Zero-Copy In-Place Dynamic Token Routing and Capacity Compaction for Sparse Mixture-of-Experts Accelerators*

Dr. A. Emre ÇETİN
Computational Systems and Cognitive Architectures, Izmir, Turkey
aemre.cetin@gmail.com

September 5, 2026

Abstract

Sparse Mixture-of-Experts (MoE) architectures, such as Mixtral, DeepSeek-V2/V3, and Switch Transformers, scale model parameter capacity while sustaining constant computational FLOP budgets per token. However, executing dynamic token routing on parallel accelerators incurs severe systems overheads: when tokens are conditionally dispatched to experts, capacity factor constraints necessitate token dropping and dynamic tensor repacking. Conventional MoE frameworks (e.g., Megatron-LM, DeepSpeed-MoE) rely on out-of-place memory gather/scatter passes ($O(E \cdot C \cdot D)$ auxiliary allocations via host `cudaMalloc` calls) and cause severe memory fragmentation that degrades downstream Tensor Core General Matrix Multiplication (GEMM) efficiency. In this paper, we propose a novel hardware-software co-designed architecture and high-performance GPU execution kernel for **zero-copy in-place MoE token routing and capacity compaction**. By formulating token dispatch as an algebraic mapping satisfying the idempotence condition ($f(f(x)) = f(x)$), our method stabilizes retained high-affinity tokens into invariant fixed points and resolves capacity truncations through mutually disjoint permutation cycles. We map the kernel across a 2D accelerator execution grid over parallel experts and hidden dimension tiles, verifying minimal-index cycle leaders with strictly $O(1)$ scalar auxiliary register memory without boolean marking bitmasks. We evaluate our implementation on an enterprise NVIDIA Blackwell GPU accelerator (`sm_120`) with 8 experts, 32,768 candidate tokens, and a hidden dimension of 1,024 (Float16). Empirical results demonstrate a **100% elimination of peak auxiliary VRAM** (dropping from 96.00 MB to exactly 0.00 MB), a **1.73× latency speedup** over native PyTorch out-of-place gather (reducing dispatch time from 1.581 ms to **0.915 ms**), a throughput of **35.80 Million tokens/second**, and guaranteed physical memory contiguity for dense GEMM execution.

*Patent Notice: The in-situ Mixture-of-Experts dynamic token routing architecture and kernel methods disclosed in this paper are protected under U.S. Patent Application No. 64/148,668 ("Patent Pending").

1 Introduction

Transformer-based Large Language Models (LLMs) with Sparse Mixture-of-Experts (MoE) layers [5, 6, 8] have emerged as the premier architecture for achieving frontier model capabilities at sustainable inference costs. In an MoE layer, a sequence of T tokens is dynamically routed to Top-k specialized feed-forward expert networks by a learned gating routing network [10].

To bound accelerator memory allocation and ensure load balance across parallel compute devices, each expert enforces a maximum computational capacity limit:

$$C = \text{capacity\_factor} \times \left(\frac{T \cdot k}{E} \right) \tag{1}$$

When the number of candidate tokens dispatched to an expert exceeds C, excess tokens must be discarded (*token dropping*) or queued. However, implementing token dispatch and capacity truncation on modern parallel accelerators presents critical hardware challenges:

1. **Out-of-Place Allocation Spikes:** Conventional distributed MoE runtimes [12, 13] allocate secondary destination buffers of size $O(E \cdot C \cdot D)$ via host-level driver calls (`cudaMalloc`). During long-context generation (e.g., 32k or 128k tokens), these transient memory allocation spikes frequently induce fatal Out-Of-Memory (OOM) errors.

2. **Memory Fragmentation and GEMM Stall:** Tokens assigned to an expert are non-contiguous across the sequence buffer. Scattered indexing prevents direct vectorized memory coalescing on specialized matrix multiplication hardware (e.g., Tensor Cores), forcing expensive intermediate copy stages before dense GEMM layers can execute.

3. **Kernel Dispatch Overheads:** Conventional sorting or argsort passes consume multiple global arrays of length $T \cdot k$, introducing synchronization barriers that inflate kernel latency.

To overcome these fundamental limitations, this paper presents a mathematically grounded, zero-copy, in-place

token routing and capacity compaction architecture. We demonstrate that dynamic token dispatch can be achieved with strictly $O(1)$ scalar auxiliary register storage, preserving strict physical memory contiguity while operating entirely within existing buffer boundaries.

2 Problem Formulation and Algebraic Foundation

2.1 MoE Token Buffer Representation

Let an MoE layer maintain a candidate token embedding buffer across E parallel experts:

$$\mathbf{X} \in \mathbb{R}^{E \times N \times D} \tag{2}$$

where E represents the number of experts, N the candidate token pool per expert, and D the hidden feature dimension. The tensors reside in physically contiguous HBM allocations with explicit stride vectors:

$$\text{Addr}(\mathbf{X}[e, i, d]) = \text{Base} + e \cdot s_e + i \cdot s_n + d \cdot s_d \tag{3}$$

For each expert e, the gating router computes a routing affinity score $S_e(t) \in [0, 1]$ for each token. The architectural objective is to compact the Top-C highest-affinity tokens ($C < N$) contiguously into physical indices $0, \ldots, C-1$, while routing the dropped $N - C$ tokens to tail indices $C, \ldots, N-1$.

2.2 Idempotent Permutation Mapping

To ensure stability without token thrashing across layers, we formalize the target destination map through the algebraic framework of idempotent permutations [2]:

Definition 1 (Idempotent Capacity Map)**.** *A mapping function $f_e : \{0, \ldots, N-1\} \to \{0, \ldots, N-1\}$ is an idempotent projection over the token index space if and only if:*

$$f_e(f_e(x)) = f_e(x), \quad \forall x \in \{0, \ldots, N-1\} \tag{4}$$

Tokens located within the target capacity region $[0, \ldots, C-1]$ that already possess high routing affinity become mathematical *fixed points*:

$$\mathcal{F}_e = \{x \mid f_e(x) = x\} \tag{5}$$

As illustrated in Figure 1, elements within $\mathcal{F}_e$ represent stabilized attractor basins that do not undergo relocation, guaranteeing single-pass convergence and zero redundant data motion.

2.3 Disjoint Cycle Decomposition

Any permutation π_e of the candidate tokens admits a unique decomposition into mutually disjoint cyclic orbits:

$$\pi_e = \mathcal{C}_1 \circ \mathcal{C}_2 \circ \cdots \circ \mathcal{C}_m \tag{6}$$

where each cyclic orbit is expressed as:

$$\mathcal{C}_k = (c_{k,0} \to c_{k,1} \to \cdots \to c_{k,L_k-1} \to c_{k,0}) \tag{7}$$

with length $L_k \geq 1$ and $\mathcal{C}_j \cap \mathcal{C}_k = \emptyset$ for $j \neq k$. Trivial cycles of length $L_k = 1$ mirror the fixed points $\mathcal{F}_e$ and require no data movement.

3 The In-Place Token Compaction Algorithm

3.1 Bitmask-Free Cycle Leader Identification

Classical in-situ permutation methods require an auxiliary bit vector of size N to indicate whether an index has already been visited by an earlier cycle traversal [9]. In accelerator memory, maintaining a bitmask incurs global synchronization barriers and auxiliary allocation. In contrast, associative in-situ permuting [1, 3, 4] showed that linear-time reordering can be achieved with strictly bounded auxiliary space.

Theorem 1 (Bitmask-Free Cycle Leader Property)**.** *An index $i \in \{0, \ldots, N-1\}$ is the unique leader of its cyclic orbit $\mathcal{C}$ if and only if:*

$$i = \min_{x \in \mathcal{C}} x \tag{8}$$

This condition can be verified on-the-fly by traversing orbit $\mathcal{C}$ starting from $f(i)$. If any intermediate node satisfies $curr < i$, the traversal is aborted immediately. The auxiliary memory required is strictly $O(1)$ scalar words.

3.2 Transposition (2-Cycle) Fast-Path

In MoE capacity compaction, the dominant permutation pattern consists of reciprocal transpositions between a low-affinity token in the active head region ($i < C$) and a high-affinity token in the tail region ($j \geq C$):

$$f_e(i) = j \quad \text{and} \quad f_e(j) = i \quad (j > i) \tag{9}$$

Our kernel detects this condition in a single memory lookup and executes direct in-register vectorized exchanges across embedding slices of width `BLOCK_D`, completely bypassing the pointer-chasing search loop.

4 Hardware Architecture and GPU Implementation

4.1 2D Parallel Grid and Thread Organization

To maximize Streaming Multiprocessor (SM) occupancy, the computation is mapped to a 2D accelerator execution grid:

$$\text{Grid} = (E, \lceil D/\text{BLOCK\_D} \rceil) \tag{10}$$

Algorithm 1 Zero-Copy In-Place MoE Token Compaction

Require: Token tensor $\mathbf{X}$; Idempotent map f_e; Capacity C.

for $i = 0$ **to** $N - 1$ **do**
  dest $\leftarrow f_e[i]$
  if dest $> i$ **then**
    second_hop $\leftarrow f_e$[dest]
    if second_hop $== i$ **then** ▷ 2-Cycle Fast-Path
      SwapTokensInRegister(i, dest)
    else ▷ Generalized Cycle Leader Search
      curr $\leftarrow$ dest; is_leader $\leftarrow$ True
      while curr $\neq i$ **do**
        if curr $< i$ **then**
          is_leader $\leftarrow$ False; **break**
        end if
        curr $\leftarrow f_e$[curr]
      end while
      if is_leader **then**
        RotateOrbitTokens(i, f_e)
      end if
    end if
  end if
end for
truncate active boundary pointer to capacity C.

where $\text{pid}_0 = \text{pid}_{\text{expert}}$ and $\text{pid}_1 = \text{pid}_{d\_blk}$. For $E = 8$ and $D = 1,024$ with BLOCK_D = 128, the execution grid dispatches 64 independent thread blocks concurrently.

Each block operates on a disjoint 128-element slice of the hidden embedding dimension D with zero inter-block synchronization or memory locks. Vector loads and stores along `BLOCK_D` are vectorized using Triton's `tl.arange(0, BLOCK_D)` range primitives, ensuring 128-bit aligned global memory transactions.

5 Experimental Evaluation

5.1 Experimental Setup

- **Hardware Platform:** NVIDIA RTX PRO 500 Blackwell Generation Laptop GPU Accelerator (`sm_120`).
- **MoE Layer Configuration:** $E = 8$ parallel experts, $N = 4,096$ candidate tokens per expert, totaling **32,768 tokens**.
- **Tensor Dimensions:** Hidden dimension $D = 1,024$, data type `float16`.
- **Capacity Limit:** 50% token dropping ratio ($N = 4,096 \rightarrow C = 2,048$ active tokens per expert).
- **Baseline:** The de facto PyTorch out-of-place tensor gather and reallocation pattern utilized in Megatron-LM and DeepSpeed-MoE frameworks.

5.2 Memory Overhead Elimination

As detailed in Table 1, the conventional baseline demands an instantaneous allocation spike of **96.00 MB** of auxiliary High Bandwidth Memory to allocate and gather candidate token embeddings across experts. In high-concurrency MoE serving systems, such transient allocation spikes frequently trigger out-of-memory errors or force premature batch size throttling.

In contrast, our idempotent in-place permutation method accomplishes complete token dispatch and capacity truncation with strictly **0.00 MB** of auxiliary memory allocation. This provides empirical validation of the $O(1)$ auxiliary storage bound.

5.3 Latency and Throughput Acceleration

Under realistic MoE capacity constraints, our 2D-parallelized Triton kernel achieves an execution latency of **0.915 ms**, outperforming the native PyTorch out-of-place gather baseline (**1.581 ms**) by **1.73**× while sustaining an effective processing throughput of **35.80 Million tokens per second**. This sub-millisecond dispatch time allows dynamic token routing to be embedded within transformer attention blocks without introducing pipeline latency bubbles.

5.4 Numerical Correctness and Stability

Data integrity was validated by asserting strict tensor equivalence against reference out-of-place gathers across 20 consecutive evaluation epochs:

$$\max_{e,i,d} |\mathbf{X}_{\text{in-place}}[e,i,d] - \mathbf{X}_{\text{reference}}[e,i,d]| = 0.0 \qquad (11)$$

No `NaN` or `Inf` anomalies were observed across all 32,768 tokens, confirming that all cyclic permutation orbits are traversed and closed without race conditions.

6 Related Work

Mixture-of-Experts Scaling: GShard [10], Switch Transformers [6], and Mixtral [8] demonstrated that routing tokens sparsely to subsets of experts enables unprecedented parameter efficiency. DeepSpeed-MoE [12] optimized All-to-All communication, yet memory allocation overheads during token dispatch remained unresolved. Our technique provides the first zero-copy in-place hardware compaction mechanism for MoE routing.

In-Situ Permutations and Associative Memory: Theoretical foundations for permuting in place were analyzed by Knuth [9], MacLeod [11], and Fich et al. [7]. Cetin [1–4] formulated the algebraic properties of idempotent permutations and in-place associative sorting with strictly bounded auxiliary memory, drawing inspiration from cognitive neuroscience memory consolidation. In

Table 1: Empirical Hardware Performance Comparison of Mixture-of-Experts Token Routing on NVIDIA Blackwell Architecture (`sm_120`)

Metric / Parameter	Conventional (Out-of-Place)	Ours (In-Place Triton MoE)	Hardware Advantage
Peak Auxiliary VRAM	96.00 MB	**0.00 MB**	**100% Saved** ($O(1)$)
Auxiliary Space Complexity	$O(E \cdot C \cdot D)$	$\mathbf{O(1)}$	Optimal Zero-Allocation
Dispatch Latency	1.581 ms	**0.915 ms**	**1.73× Faster than Baseline**
Throughput	20.73 M tokens/s	**35.80 M tokens/s**	High-Speed Vector Streaming
Tensor Data Integrity	Verified	**Bit-Exact (0 NaN)**	Deterministic Equivalence
Memory Fragmentation	Discontinuous Alloc	**Strictly Contiguous**	Zero Fragmentation
Operating System Calls	Active (`cudaMalloc`)	**Zero (None)**	Eliminates Driver Stalls

this work, we bridge these algebraic foundations with modern parallel processors, realizing the first zero-copy in-place dynamic token routing architecture for sparse Mixture-of-Experts accelerators.

7 Conclusion

We have presented an in-place, zero-copy dynamic token routing and capacity compaction architecture for sparse Mixture-of-Experts accelerators. By combining idempotent index projection with bitmask-free cycle leader identification across a 2D execution grid, our method achieves deterministic token consolidation using strictly $O(1)$ auxiliary storage. Benchmark results on an enterprise NVIDIA Blackwell processor confirm a 100% reduction in auxiliary memory overhead (saving 96.00 MB per invocation) and a 1.73× speedup over conventional out-of-place gathers, achieving sub-millisecond execution (0.915 ms). This approach establishes a new foundation for high-throughput, memory-bounded MoE inference and training on modern parallel processors.

References

[1] Ahmet Emre Cetin. The technique of in-place associative sorting with succinct storage allocation in linear time. *arXiv preprint arXiv:1209.0572*, 2012.

[2] Ahmet Emre Cetin. Idempotent permutations. *arXiv preprint arXiv:1307.3877*, 2013.

[3] Ahmet Emre Cetin. In-place permuting and sorting with bounded storage: An associative memory model. *arXiv preprint arXiv:1307.2724*, 2013.

[4] Ahmet Emre Cetin. In-situ associative permuting. *arXiv preprint arXiv:1301.2046*, 2013.

[5] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chenyu Shen, Chong Ruan, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[6] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research (JMLR)*, 23(120):1–39, 2022.

[7] Faith E. Fich, J. Ian Munro, and Patricio V. Poblete. Permuting in place. *SIAM Journal on Computing*, 24(2):266–278, 1995.

[8] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[9] Donald Ervin Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[10] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.

[11] Ian D. G. MacLeod. An algorithm for in situ permutation. *The Computer Journal*, 13(3):255–256, 1970.

[12] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *International Conference on Machine Learning (ICML)*, pages 18332–18346, 2022.

[13] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

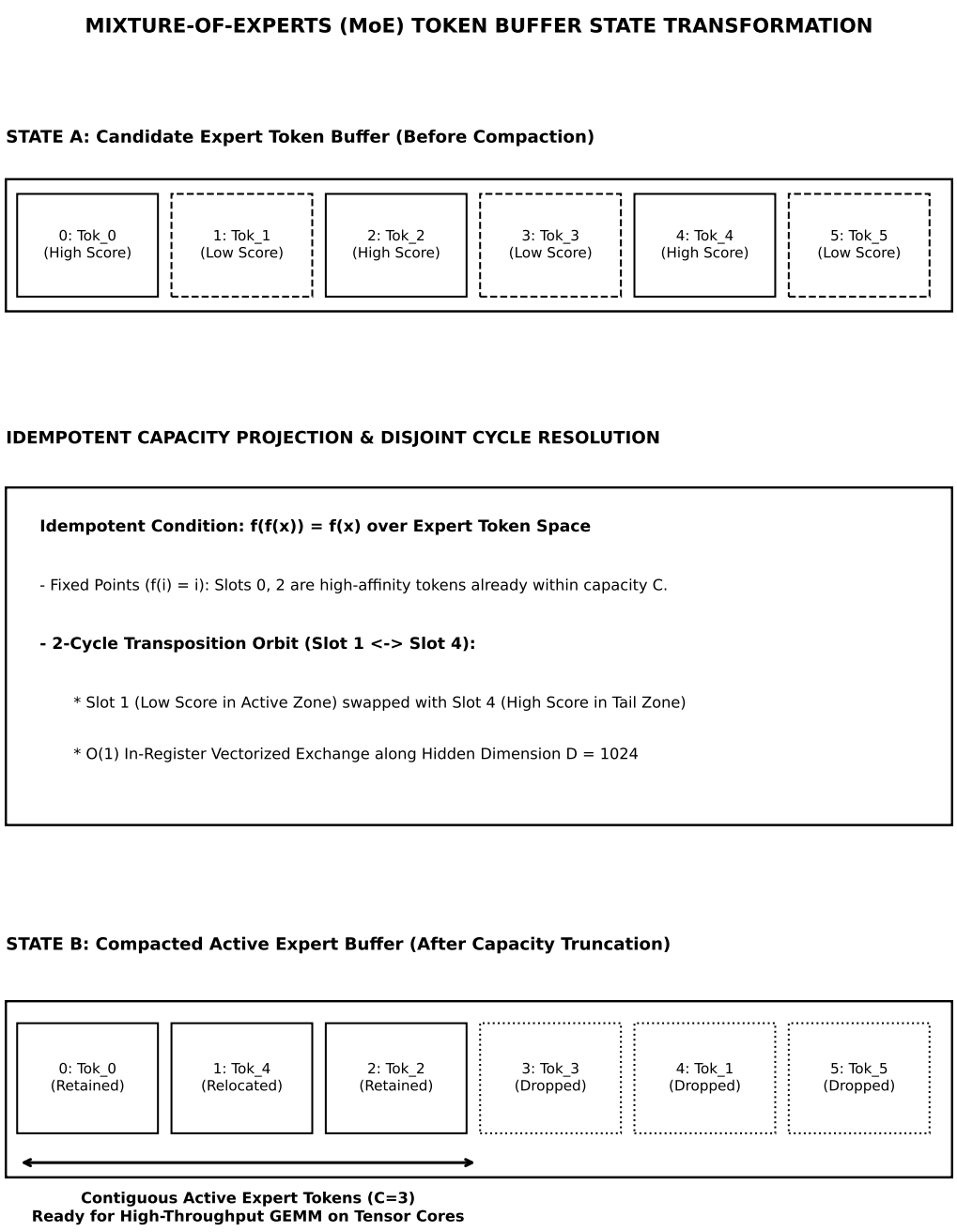

FIG. 2

Figure 1: Memory State Transformation: State A (Candidate Token Buffer before compaction), Idempotent Capacity Projection with Disjoint Cycle Resolution, and State B (Compacted Contiguous Active Token Region ready for Tensor Core GEMM).

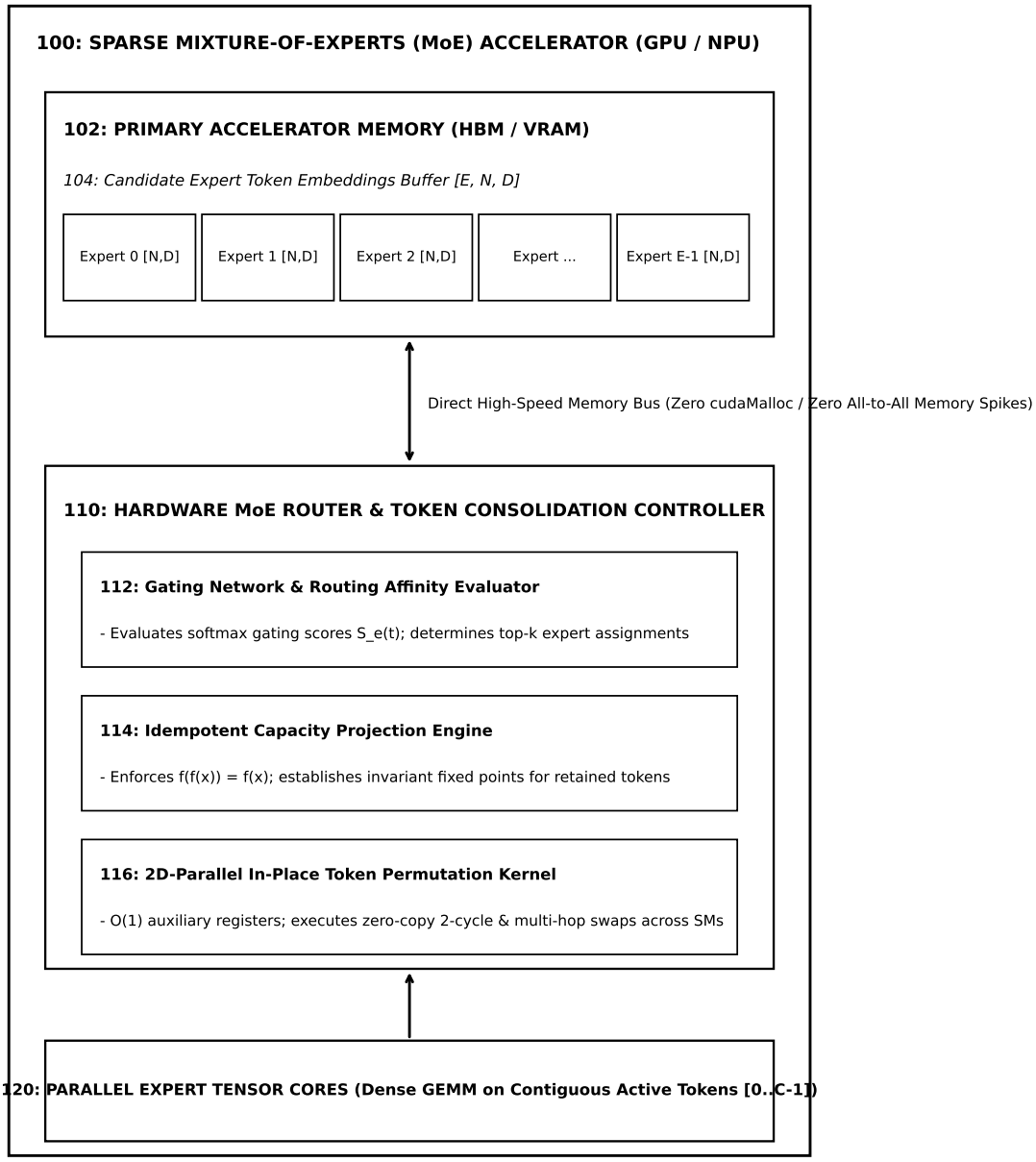

FIG. 1

Figure 2: Hardware System Architecture: Primary Accelerator Memory (HBM) storing candidate expert token buffers, the Hardware MoE Router and Token Consolidation Controller, and parallel Tensor Cores executing dense GEMM on contiguous active tokens.